



# COMPSCI 130 S1 2022

## Introduction to Software Fundamentals

Priority Queues – Implementation 2



# Agenda & Reading

---

- ▶ **Agenda**
  - ▶ PriorityQueue class
    - ▶ delete\_minimum()
- ▶ **Reading**
  - ▶ Online Textbook: <https://onlinetextbook.auckland.ac.nz/20-priority-queues-and-heaps/>

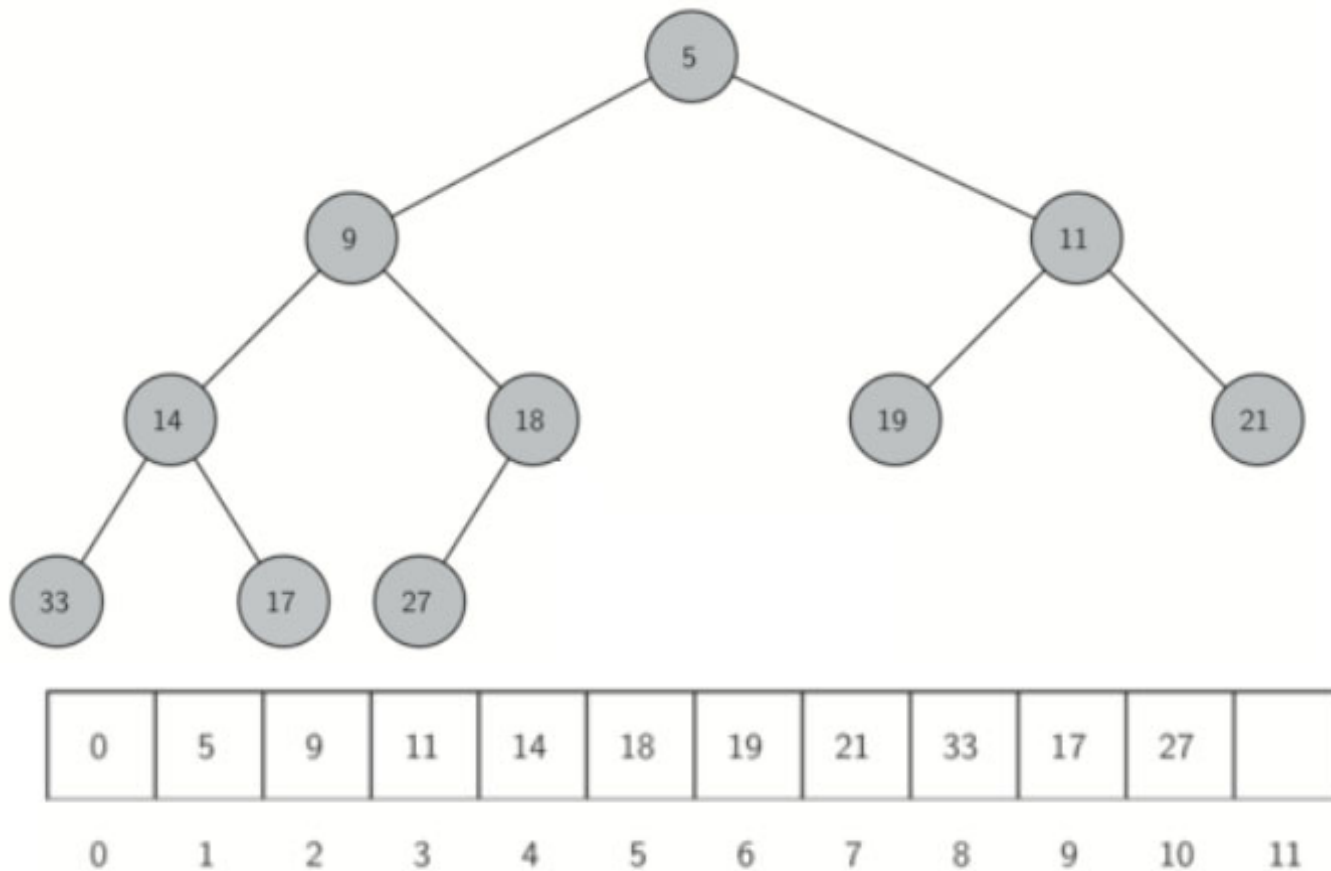




# Heap operations – delete\_minimum()

- Deletion : **priority\_queue.delete\_minimum()**

The minimum is **always** the root!

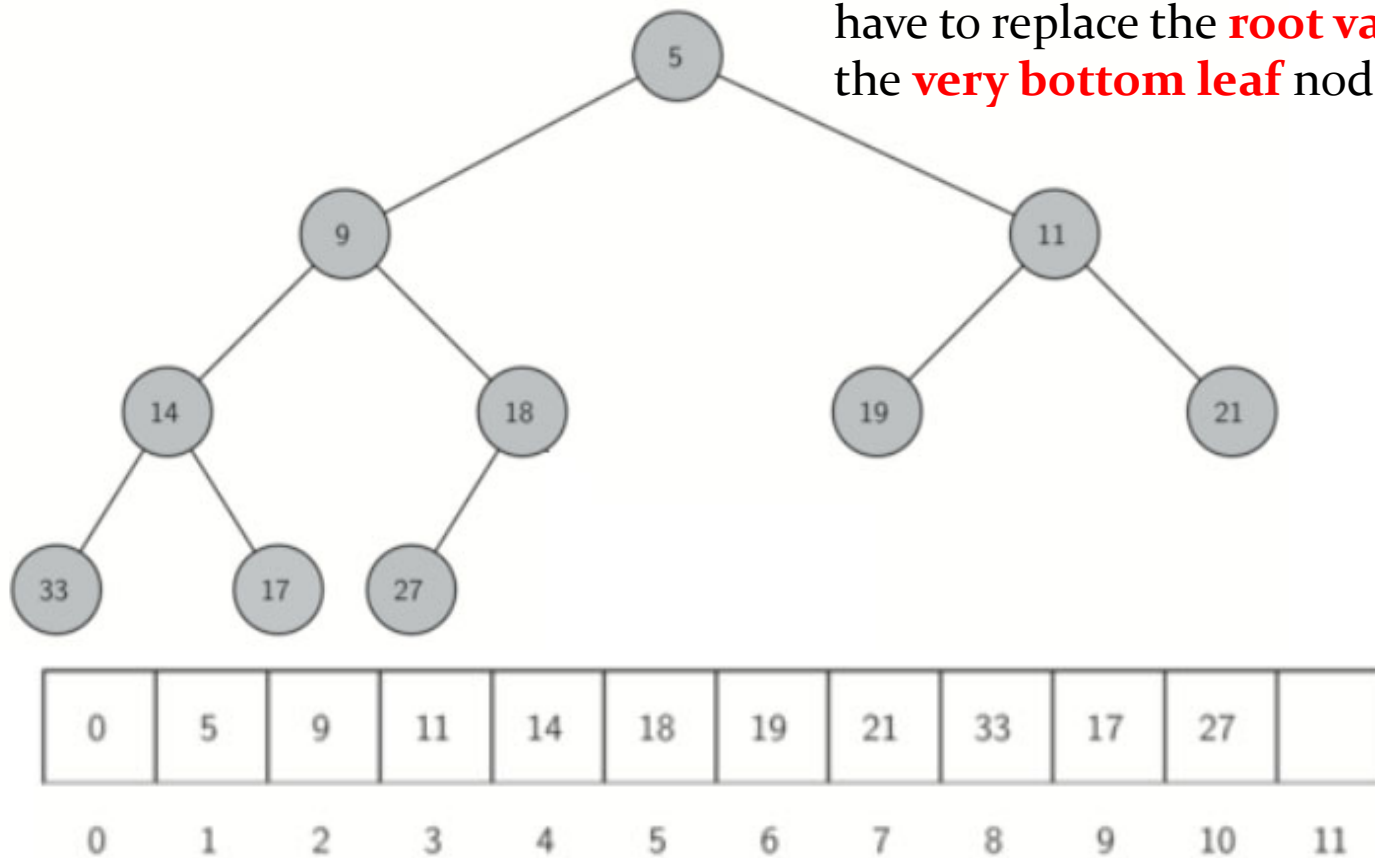




# Heap operations – delete\_minimum()

- Deletion : **priority\_queue.delete\_minimum()**

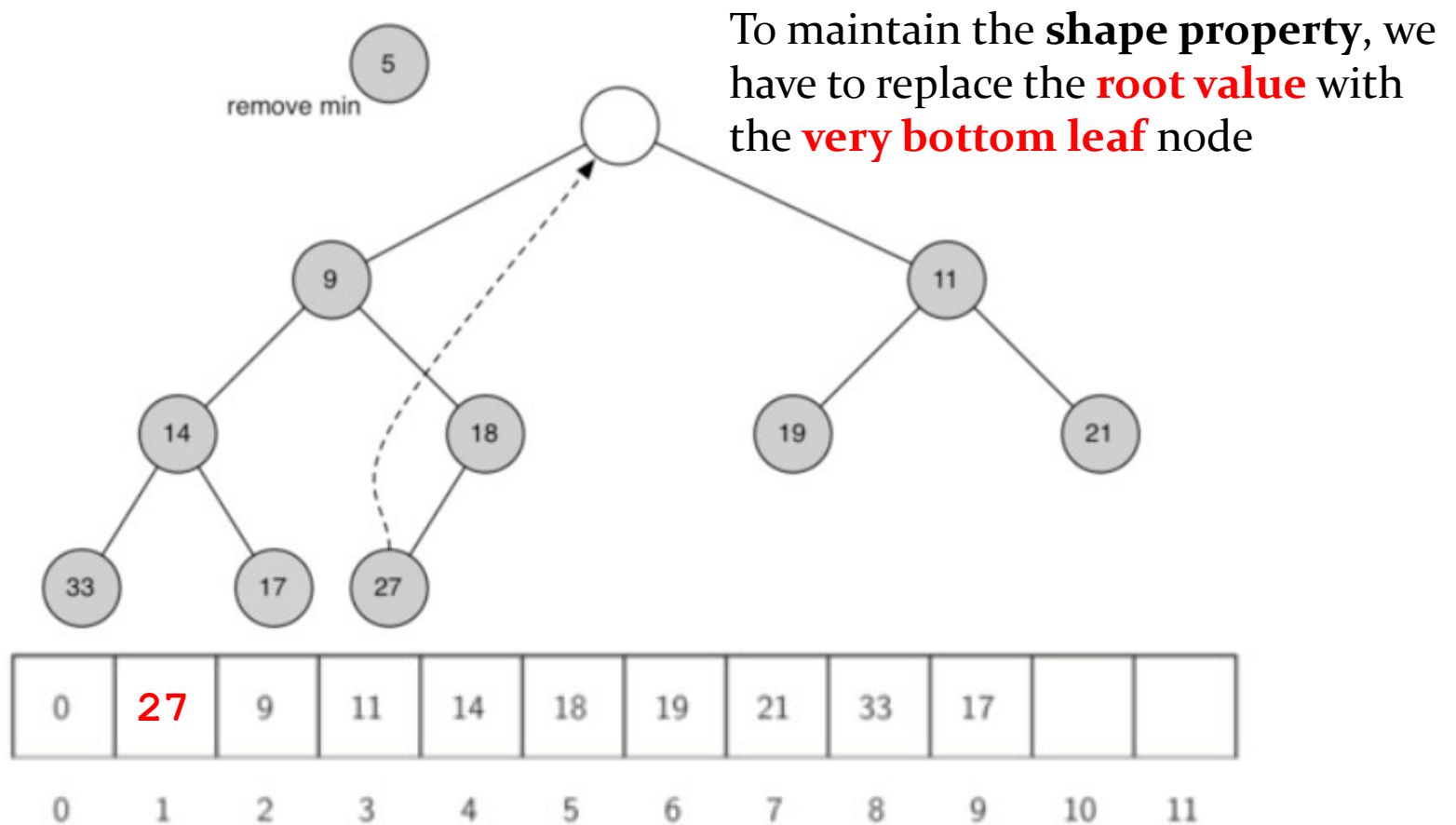
To maintain the **shape property**, we have to replace the **root value** with the **very bottom leaf** node





# Heap operations – delete\_minimum()

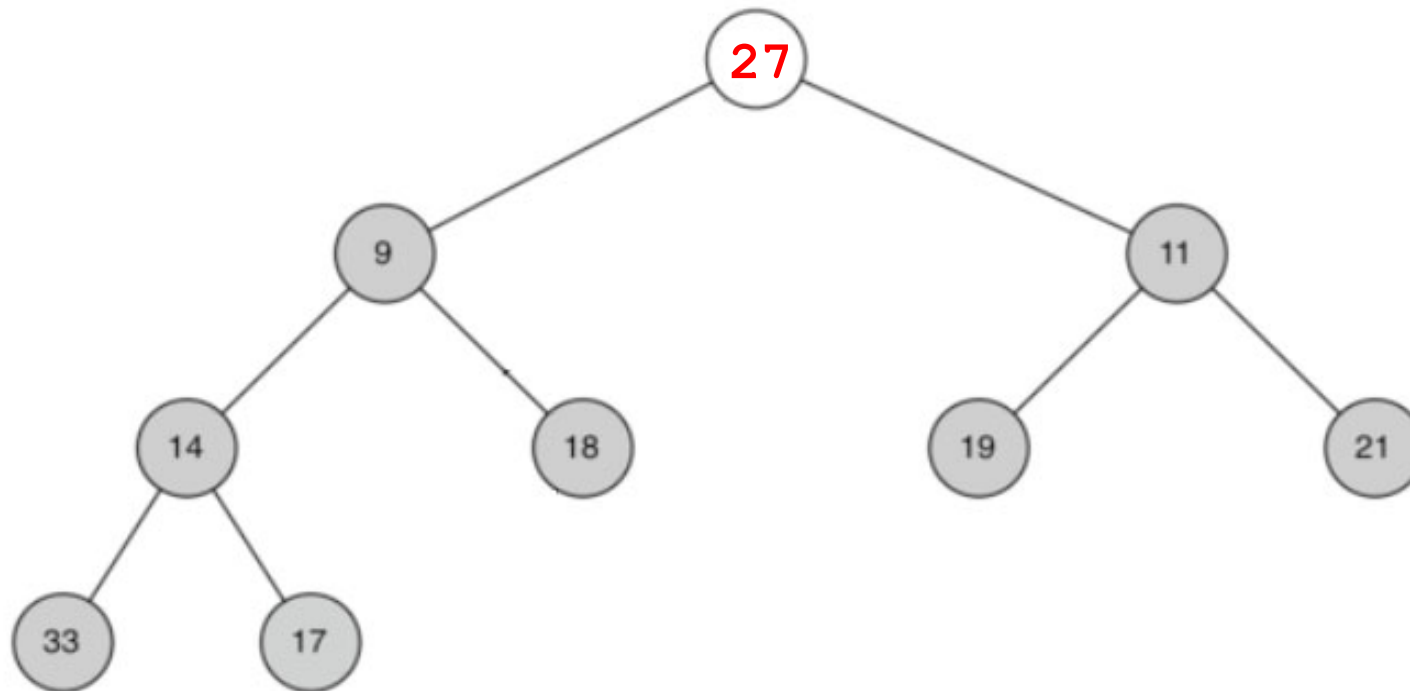
- Deletion : **priority\_queue.delete\_minimum()**





## Heap operations – delete\_minimum()

- Deletion : **priority\_queue.delete\_minimum()**

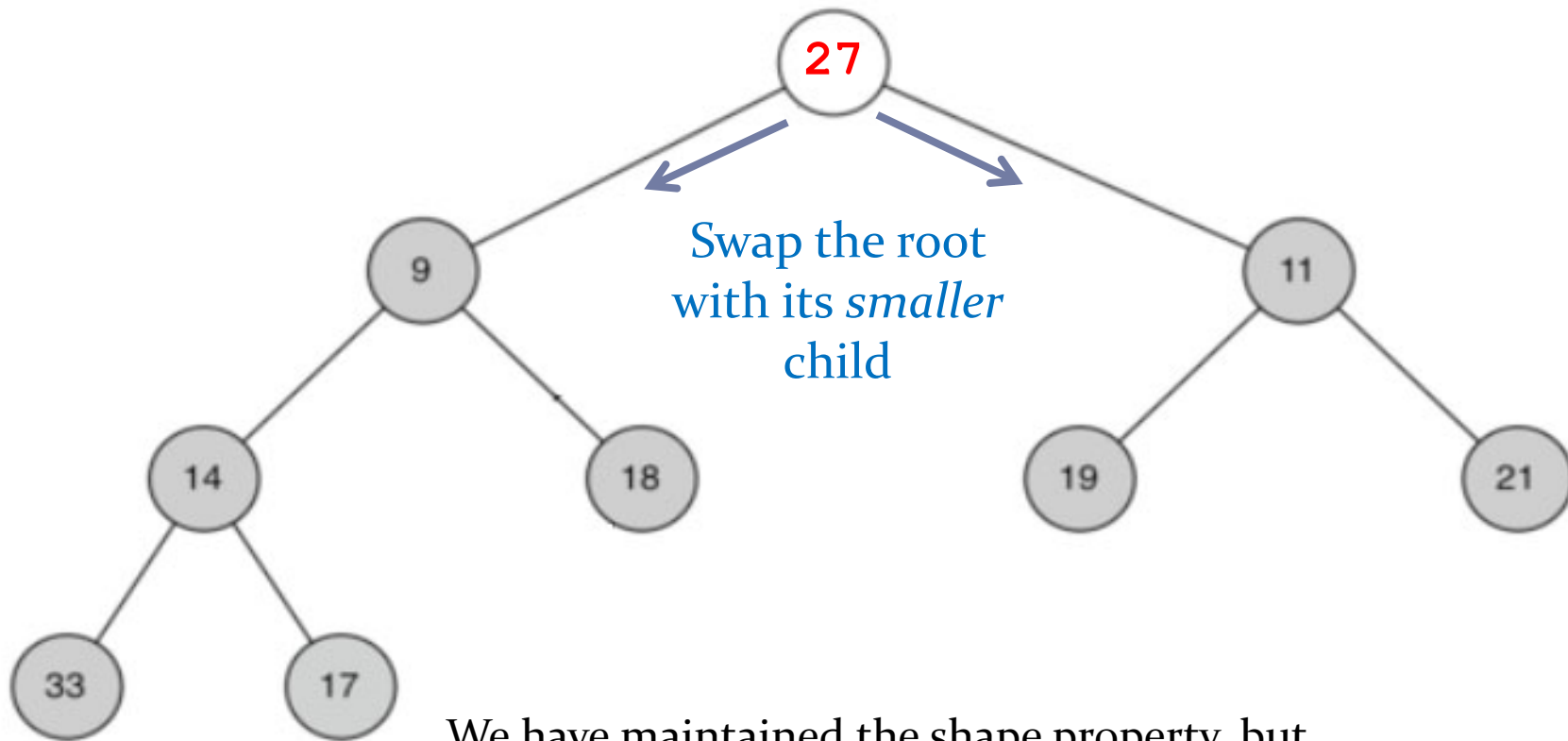


We have maintained the shape property, but now we have broken the **order property**.  
*How can we restore it?*



## Heap operations – delete\_minimum()

- Deletion : **priority\_queue.delete\_minimum()**

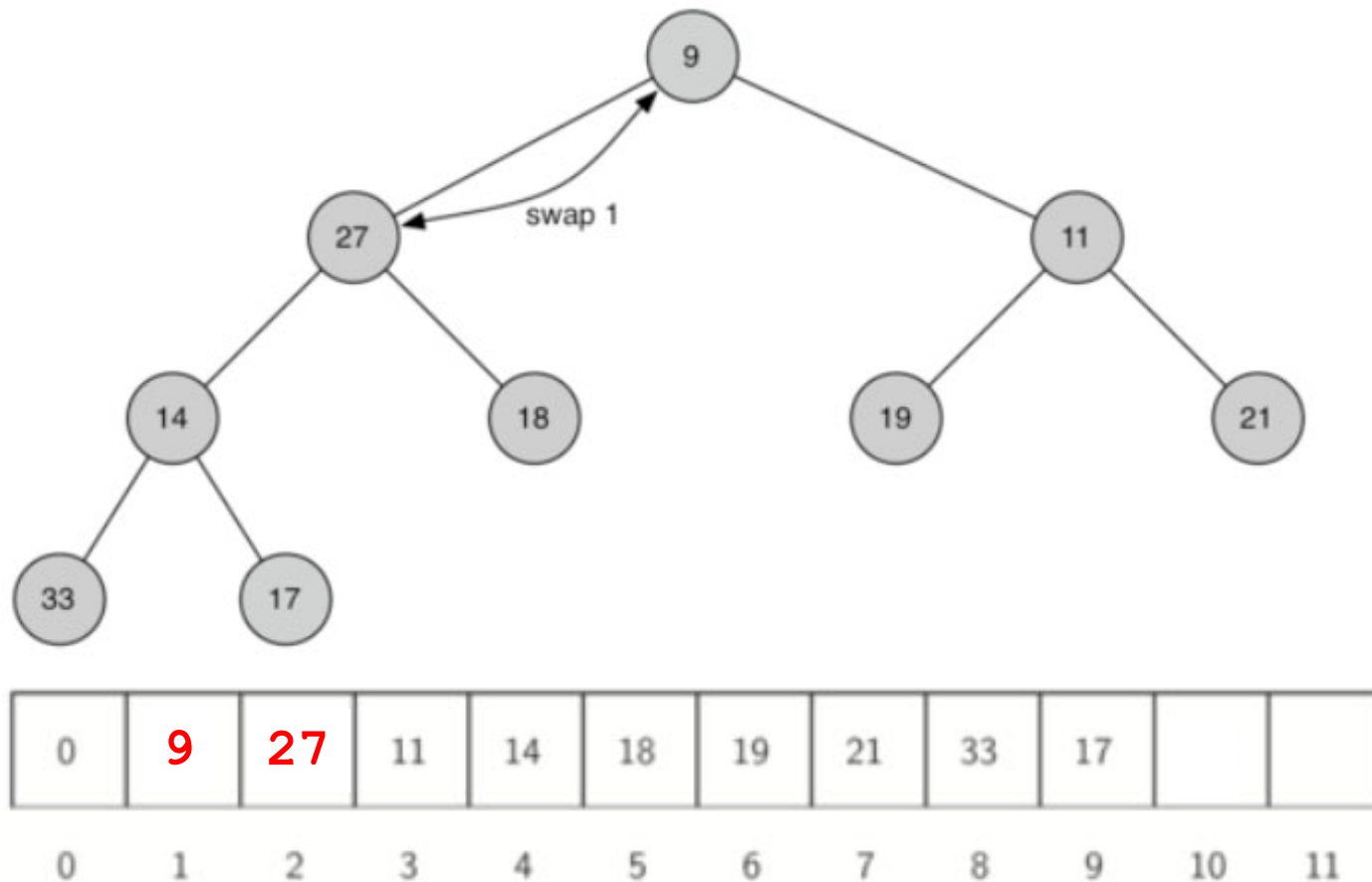


We have maintained the shape property, but now we have broken the **order property**.  
*How can we restore it?*



# Heap operations – delete\_minimum()

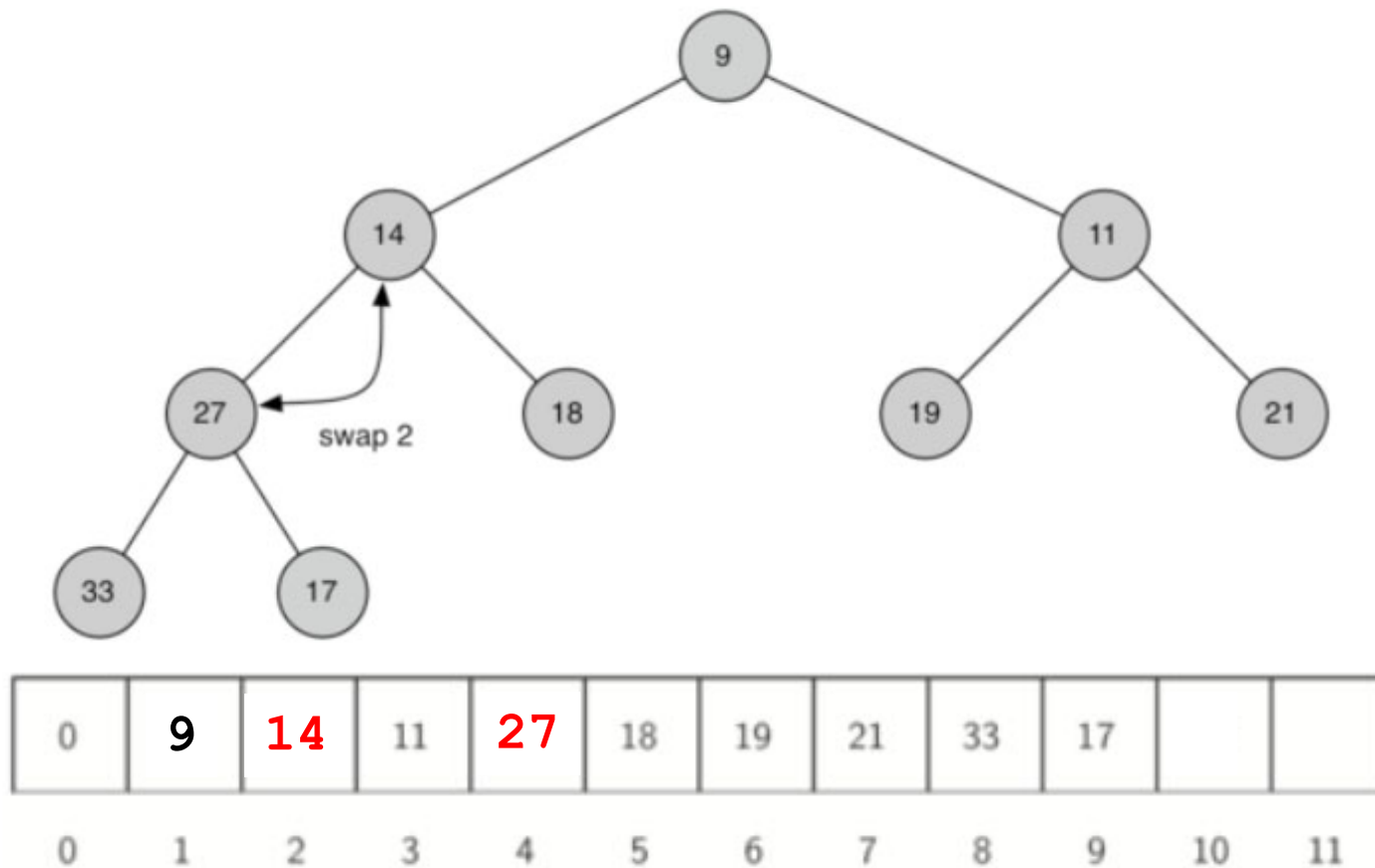
- Deletion : **priority\_queue.delete\_minimum()**





# Heap operations – delete\_minimum()

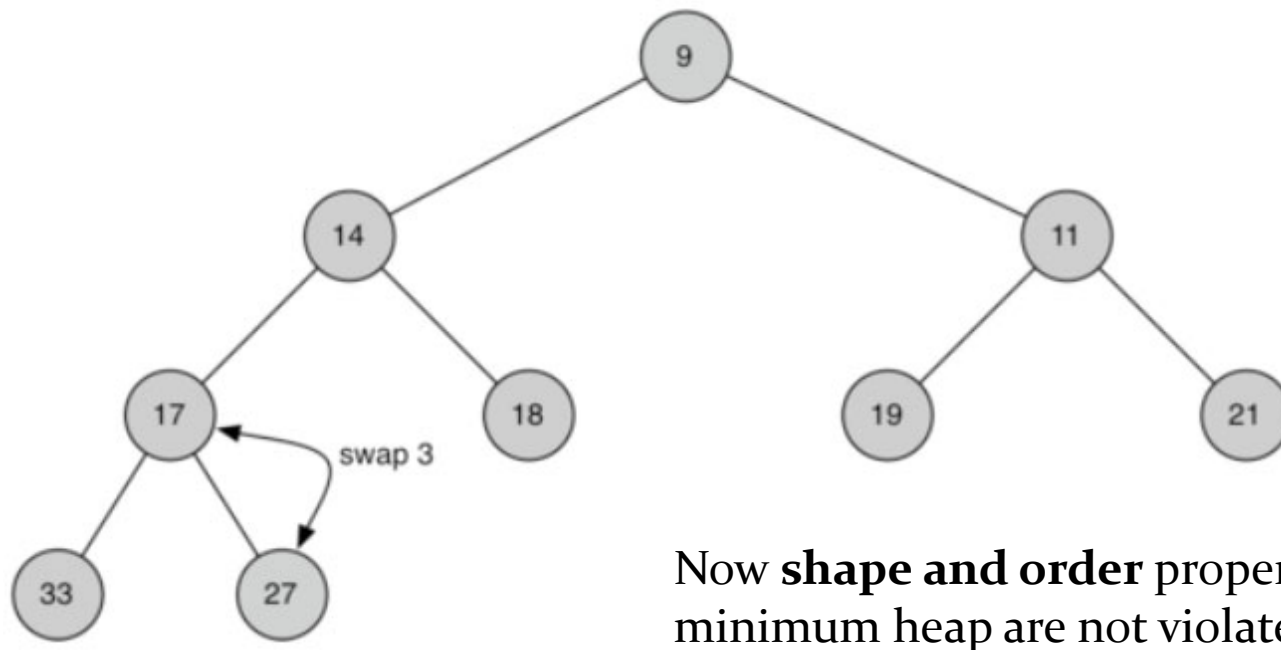
- Deletion : **priority\_queue.delete\_minimum()**





## Heap operations – delete\_minimum()

- Deletion : **priority\_queue.delete\_minimum()**



|   |   |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|
| 0 | 9 | 14 | 11 | 17 | 18 | 19 | 21 | 33 | 27 |    |    |
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |



# Heap operations – delete\_minimum()

---

```
def percolate_down(self, index):
    while (index * 2) <= self.size:
        // complete this

        index = smallest_child

def get_smaller_child_index(self, index):
    if index * 2 + 1 > self.size:
        return index * 2
    else:
        if self.binary_heap[index*2] < self.binary_heap[index*2+1]:
            return index * 2
        else:
            return index * 2 + 1

def delete_minimum(self):
    minimum_value = self.binary_heap[1]
    self.binary_heap[1] = self.binary_heap[self.size]
    self.binary_heap.pop()
    self.size -= 1
    self.percolate_down(1)
    return minimum_value
```



# Priority Queues

## ► Complexity

- for a priority queue with  $n$  items, the height (depth) of the binary heap is at most  $\log(n)$
- therefore, binary heap operations are  $O(\log n)$

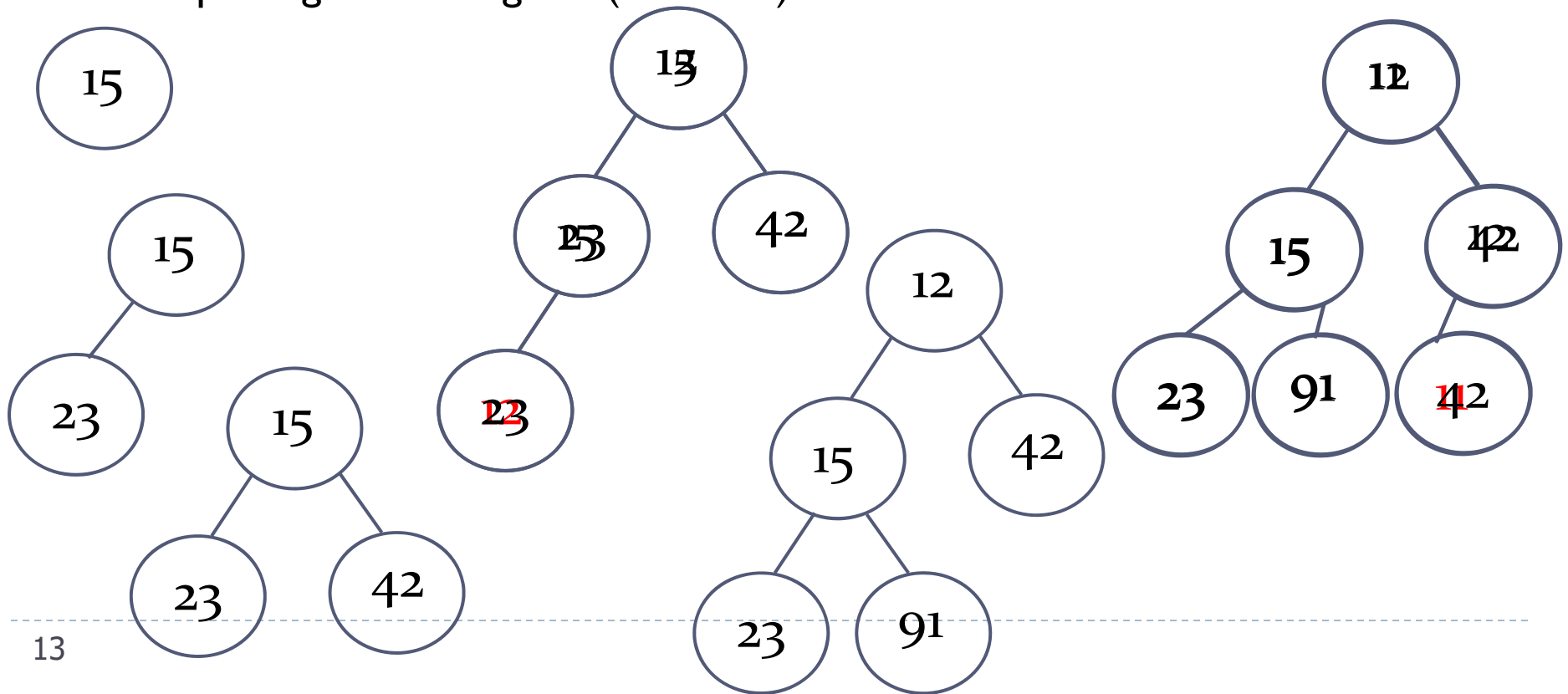
| Data structure                      | Insert item                   | Remove min/max                |
|-------------------------------------|-------------------------------|-------------------------------|
| <b>Unsorted</b> array / Python list | $O(1)$                        | $O(n)$                        |
| <b>Sorted</b> array / Python list   | $O(n)$                        | $O(1)$                        |
| <b>Unsorted</b> linked list         | $O(1)$                        | $O(n)$                        |
| <b>Sorted</b> linked list           | $O(n)$                        | $O(1)$                        |
| Hashtable                           | $O(1)$                        | $O(n)$                        |
| Binary search tree                  | $O(\log n)$                   | $O(\log n)$                   |
| <b>Binary heap</b>                  | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> |

But, for binary heap we get **guaranteed**  $O(\log n)$  performance, lower memory requirements and a much simpler implementation... and we can **peek()** in  $O(1)$



# Example

- ▶ Draw the heap structure after inserting the following integer search key values (in the order given) into an empty heap:
  - ▶ 15, 23, 42, 12, 91, 11
- ▶ Show the structure of the heap after EACH insertion. NOTE: show the state of the heap using a tree diagram (not a list).



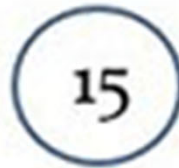


# Example

---

- Insert **15**, 23, 42, 12, 91, 11

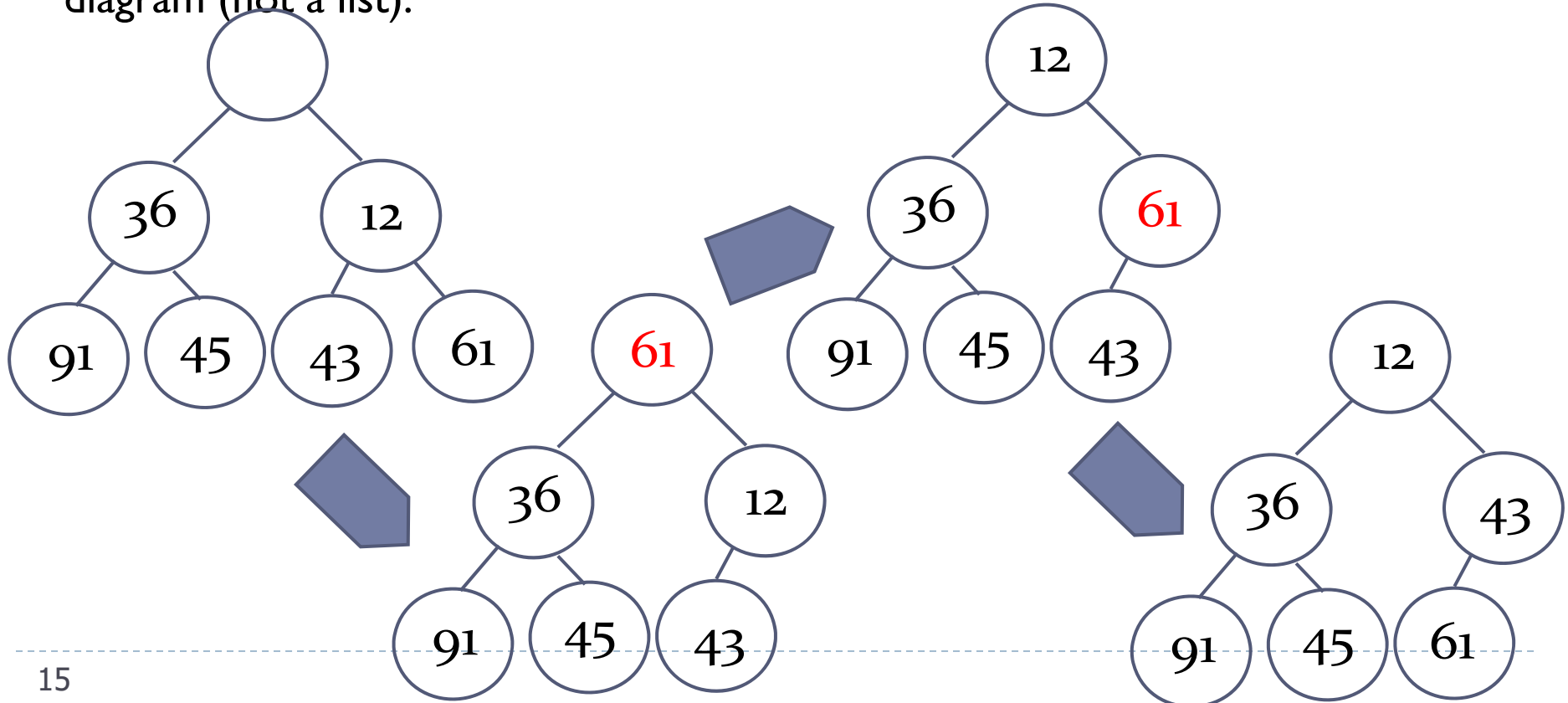
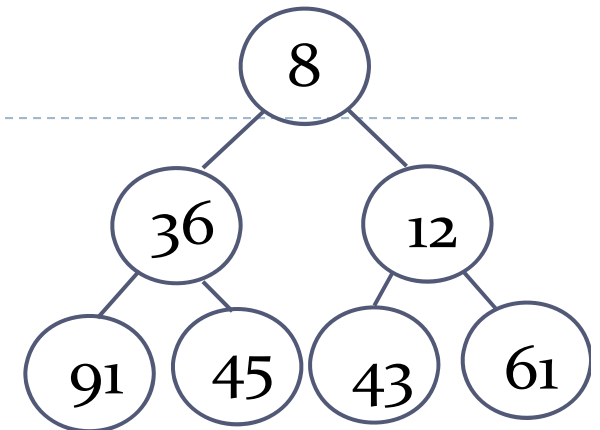
insert 15





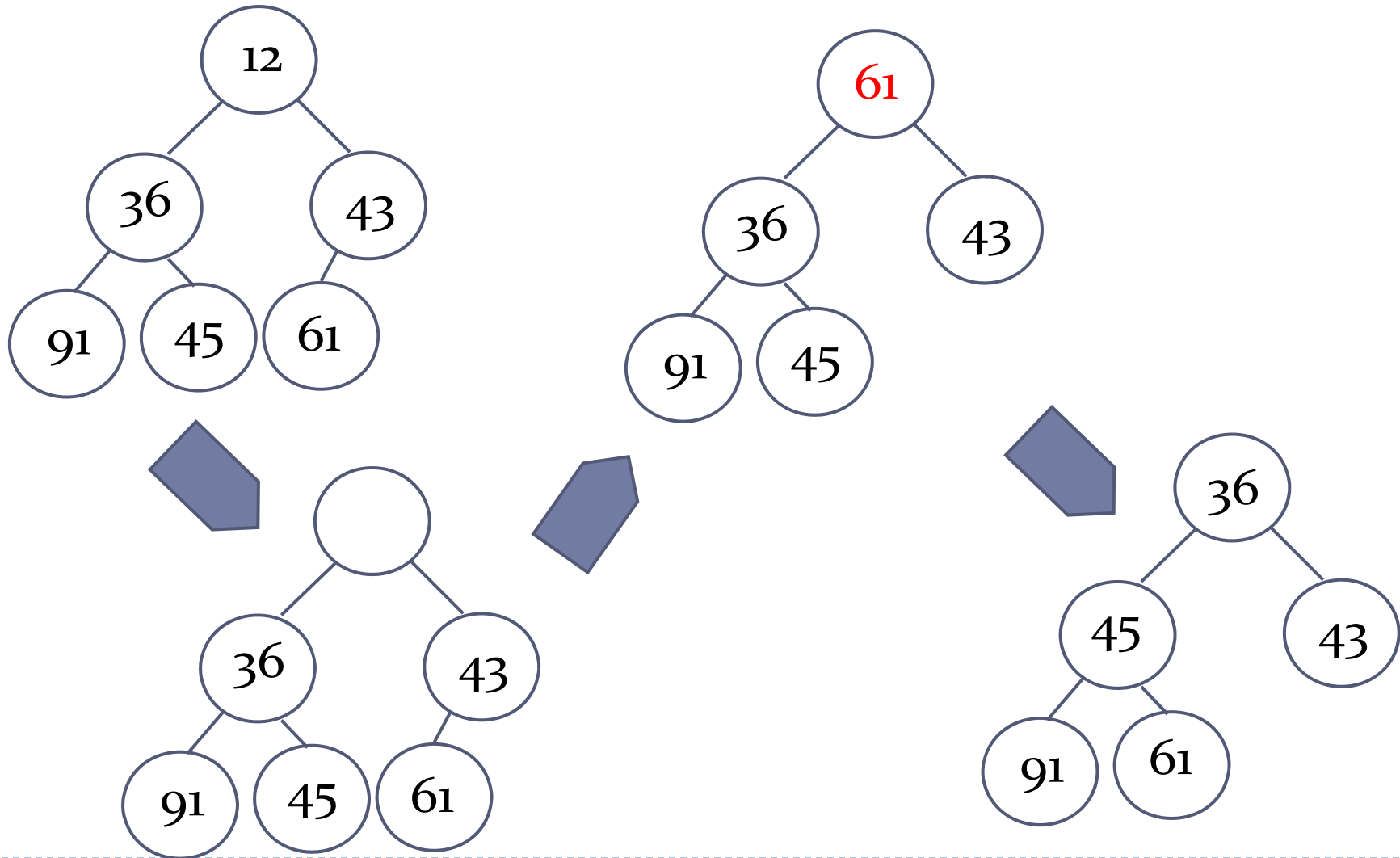
# Heap Delete

- ▶ Given is the binary heap on the right. Draw the heap after calling **delete\_minimum()** twice.
- ▶ Show the structure of the heap after each deletion.  
NOTE: show the state of the heap using a tree diagram (not a list).



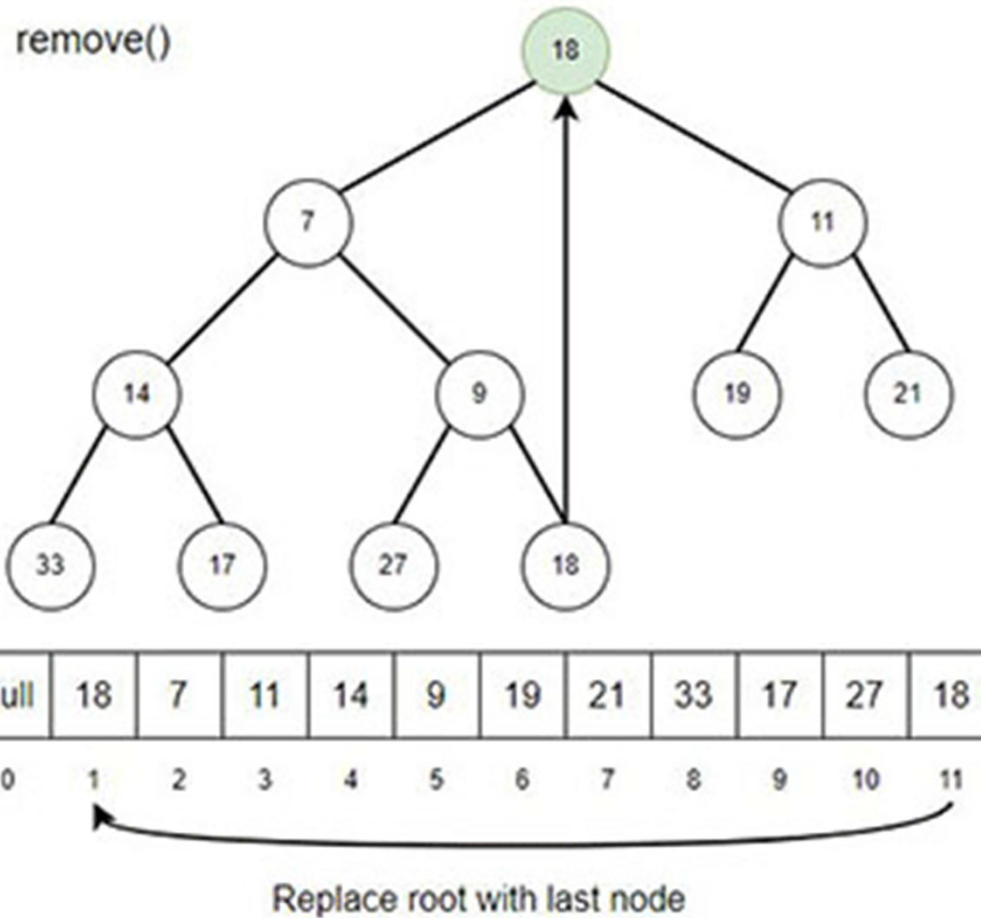


# Heap Delete (continued)





# Delete



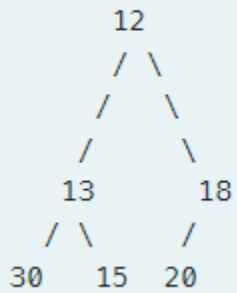
This is an animation!



## Exercise 8

---

Given the binary heap (minimum-heap) below, what does the heap look like after removing the root?





# Building a Binary Heap

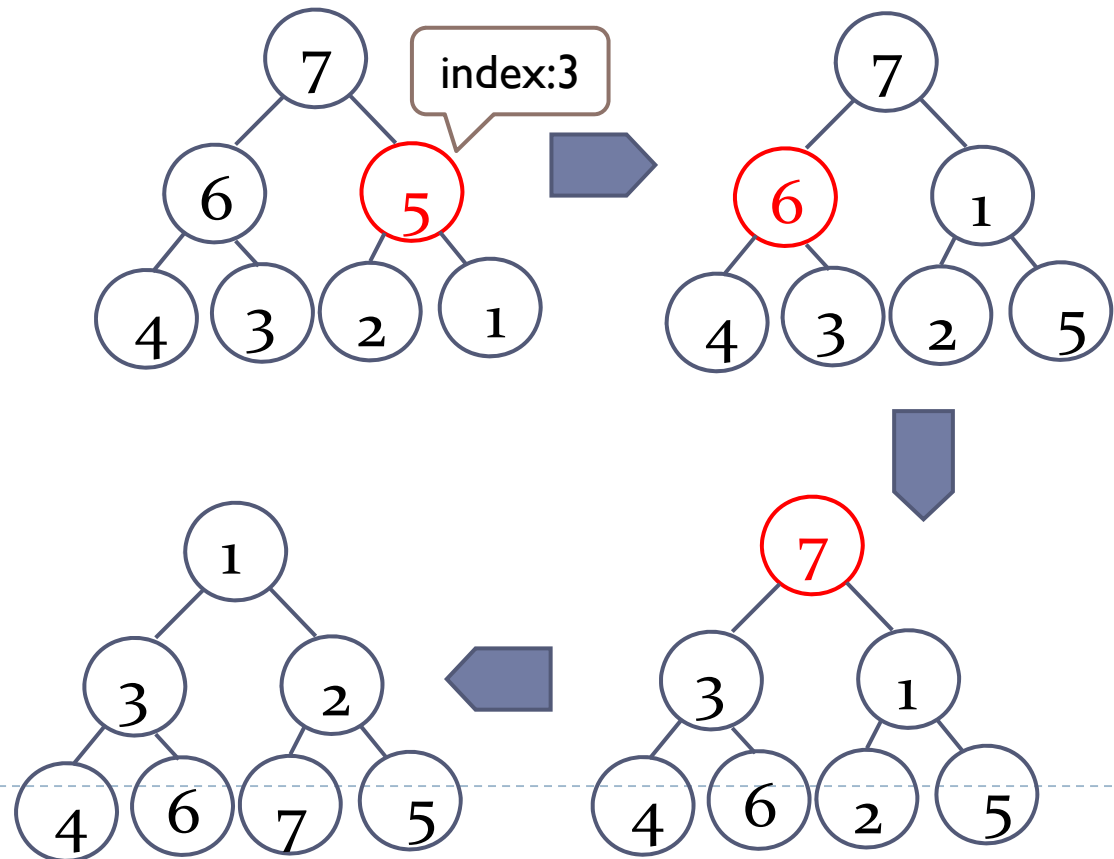
---

- ▶ There are two different methods to build a binary heap from a list of values. We can either:
  - ▶ Add the list of values to an empty heap one by one. The complexity is  **$O(n \log n)$**
  - ▶ Insert all values into the `binary_heap` list and rearrange the elements starting from the last element which has a child (i.e. `size // 2`) and keep rearranging by calling the `percolate down` method and working backwards towards the root. The complexity of this method is  **$O(n)$** . The method is called “heapify”



## Heapify – $O(n)$

- ▶ Convert the [7, 6, 5, 4, 3, 2, 1] into a binary heap
- ▶ Rearranging by calling the percolate down method and working backwards towards the root.
  - ▶ starting at size // 2



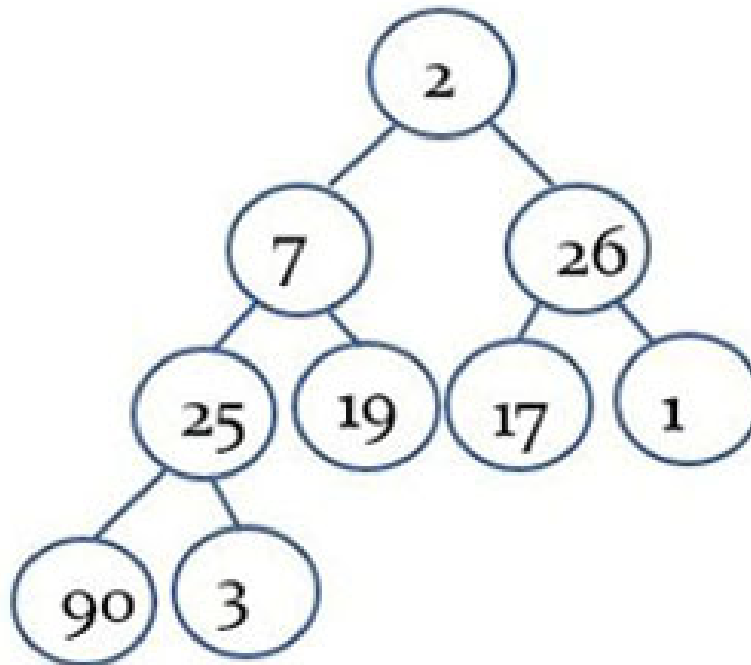


## Heapify – $O(n)$

---

- ▶ starting at index: 4, element 25

Insert 2, 7, 26, 25, 19, 17, 1, 90, 3

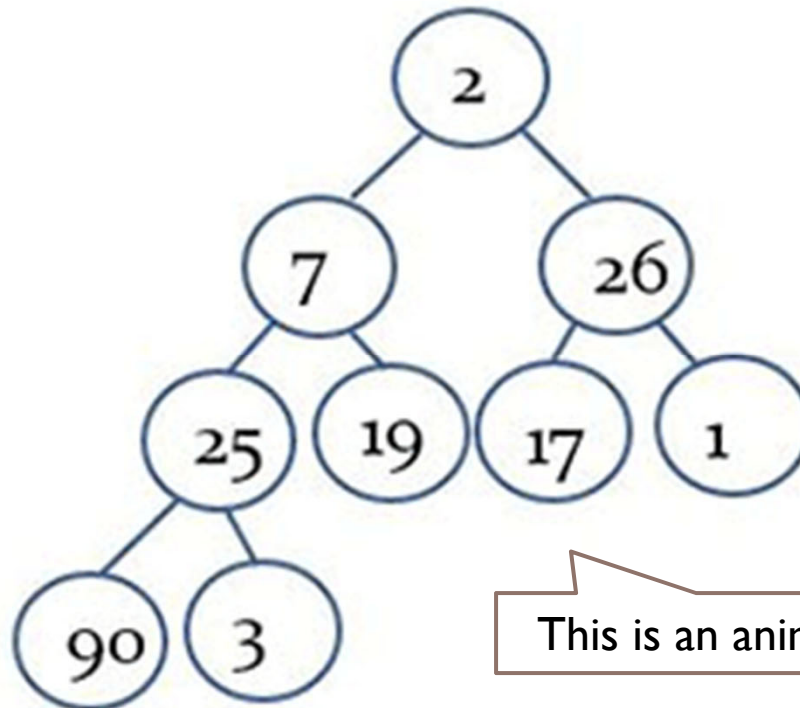




# Animation

- ▶ starting at index: 4, element 25

Insert 2, 7, 26, 25, 19, 17, 1, 90, 3



This is an animation!